

ML Weekend Project Plan

IRIS-style World Model + KV-Cache Decode Attention Kernel

Portfolio Project

3 Full Weekends

PyTorch + Triton

Cluster Required

Project Overview

Build an IRIS-style tokenized world model on Atari Breakout, then implement a tiled KV-cache decode attention kernel in Triton that accelerates the autoregressive imagination loop. The world model provides the *what* — a system worth optimizing — and the kernel targets the central bottleneck in its generation: loading the full KV-cache from HBM on every single token step. This is the same problem every serious LLM inference framework has had to solve, making it immediately legible and impressive to systems interviewers.

What You Ship

- **A trained world model** — VQ-VAE tokenizer + autoregressive transformer generating imagined Atari rollouts, with reconstruction metrics (MSE, FID, SSIM) and codebook health metrics
- **A tiled KV-cache decode attention kernel** — written in Triton, keeps KV-cache in SRAM tiles to minimize HBM round-trips during single-token autoregressive decoding
- **A rigorous benchmark table** — tokens/sec and per-step latency vs PyTorch baseline across sequence lengths $L = [128, 256, 512, 1024]$ on A100, showing speedup scaling with L
- **A clean repo and README** — architecture diagram, imagined rollout GIFs, benchmark table, and honest metric reporting

Architecture

Stage 1 — VQ-VAE Tokenizer

A CNN encoder compresses each 64×64 grayscale Atari frame into a 4×4 spatial grid of latent vectors (16 positions total). A vector quantizer maps each position to its nearest entry in a learned codebook of $K=512$ discrete codes via nearest-neighbour lookup. A CNN decoder reconstructs the frame from the quantized codes. After training, each frame is represented as 16 integer tokens drawn from a vocabulary of 512 — turning visual prediction into a language modelling problem.

Stage 2 — Transformer World Model

A GPT-style transformer (~20M params) operates on sequences of tokens interleaved with actions: `[frame1_tokens, action1, frame2_tokens, action2, ...]`. Trained autoregressively to predict the next

token. At inference, the model generates imagined future frames one token at a time — this autoregressive loop is where the KV-cache kernel lives.

Data Flow

```
Real frame → VQ-VAE encoder → 16 discrete tokens → Transformer predicts next tokens
→ VQ-VAE decoder → Imagined frame
```

The Kernel: KV-Cache Decode Attention

During autoregressive generation the transformer processes one new token at a time. Standard attention reloads the entire KV-cache from HBM (GPU high-bandwidth memory) on every single step — even though only one new query is being processed. This is severely memory-bandwidth bound and gets worse as sequence length grows.

A tiled decode attention kernel fixes this by loading the KV-cache in SRAM tiles and computing the attention score + weighted sum in one fused pass, dramatically reducing HBM round-trips. Speedup scales with sequence length — the longer the imagined rollout, the more the kernel helps.

Naive vs Fused — Memory Passes

```
Naive PyTorch per decode step:
```

```
load Q (new token) → load full K cache → load full V cache → compute scores → write
output = 3 full HBM reads per step
```

```
Triton tiled kernel:
```

```
tile K+V into SRAM → compute attention + weighted sum in one pass → write output = 1
pass, minimal HBM round-trips
```

3-Weekend Plan

Each weekend assumes ~12–16 hours of focused work (full days, Saturday + Sunday).

Weekend	Day	Goal	End State
Weekend 1	Sat	Data collection via Gymnasium (random agent, Breakout), preprocess to 64×64 grayscale, store as HDF5	100k+ frames on disk
	Sun	Train VQ-VAE with EMA codebook + random restart + k-means init. Monitor perplexity and codebook utilization	Working tokenizer, perplexity >60% of K

	Sat	Train transformer world model on token sequences. Get first imagined rollouts generating. Log to W&B;	Imagined frames generating
Weekend 2	Sun	Profile autoregressive generation loop with torch.profiler. Confirm KV-cache attention is the bottleneck. Read Triton docs, implement naive correct kernel	Correct kernel (not yet fast)
	Sat	Add SRAM tiling to kernel. Benchmark across L = [128, 256, 512, 1024] with proper CUDA warmup and synchronization	Speedup numbers measured
Weekend 3	Sun	Compute test metrics (MSE, FID, SSIM). Polish README with GIFs, architecture diagram, benchmark table. Push to GitHub	Shippable portfolio piece

Key risk: Weekend 2 has highest variance. If profiling shows KV-cache attention is not the top bottleneck, pivot to whatever torch.profiler shows at the top of the trace. Always let the profiler pick your target — never guess.

Metrics To Track

VQ-VAE — Training & Validation

Metric	Split	Frequency	Why
recon_loss (MSE)	train + val	Every step / epoch	Primary learning signal
vq_loss	train	Every step	Codebook health
commitment_loss	train	Every step	Diagnose collapse early
codebook_perplexity	train + val	Every step	Uniformity of code usage — best collapse indicator
codebook_utilization	train + val	Every 100 steps	% of codes used over rolling window
Reconstruction images	val	Every 500 steps	Visual sanity check — most important diagnostic

VQ-VAE — Test Set (run once at end)

- **FID** — Fréchet Inception Distance. Measures distributional similarity of reconstructed vs real frames using InceptionV3 features. Lower is better. Requires 10k+ samples.
- **SSIM** — Structural Similarity Index. Perceptual similarity metric accounting for luminance, contrast, and structure. Range 0–1, higher is better.
- **Test recon_loss (MSE)** — pixel-level reconstruction accuracy on held-out frames.

Kernel Benchmark Table

Measure per-step decode latency (ms) and throughput (tokens/sec) at each sequence length with N=100 timed iterations after 20 warmup iterations, with torch.cuda.synchronize() before and after each measurement:

Sequence Length (L)	PyTorch (ms/step)	Triton Kernel (ms/step)	Speedup
128	X	X	~1.5–3×
256	X	X	~2–4×
512	X	X	~3–6×
1024	X	X	~4–8×

Fill in X values from your actual runs. Speedup should increase with L — if it doesn't, the kernel isn't reducing HBM round-trips effectively.

Resume & Portfolio

Project Line

Tokenized World Model + KV-Cache Decode Attention Kernel (PyTorch, Triton) – github.com/you/repo

Bullet Points

- Trained an IRIS-style world model (VQ-VAE + [X]M param transformer) on [N]k Atari Breakout frames, generating [T]-step imagined rollouts with [X] FID and [X] SSIM on held-out frames
- Implemented a tiled KV-cache decode attention kernel in Triton for single-token autoregressive decoding, achieving [X]× reduction in per-step latency at L=512 on A100 by minimizing HBM round-trips — speedup scales with sequence length
- Benchmarked fused kernel against PyTorch baseline across L = [128, 256, 512, 1024] with proper CUDA synchronization and warmup; documented full results in reproducible benchmarking suite

Fill in bracketed values from your actual measurements. Never put a number in a resume bullet you can't defend precisely — interviewers will ask batch size, sequence length, warmup iterations, and baseline definition.

Skills to List

Category	Skills
Frameworks	PyTorch, Triton, Gymnasium
Concepts	VQ-VAE, discrete tokenization, autoregressive transformers, KV-cache, tiled attention, HBM vs SRAM memory hierarchy, model-based RL

Systems	GPU memory profiling, CUDA synchronization, HBM bandwidth optimization, mixed precision (bf16), SLURM
Tools	Weights & Biases, torch.profiler, NSight, torch.compile

Background Knowledge

VQ-VAE

A Vector Quantized VAE maps images to discrete tokens using a learned codebook. The encoder produces continuous latent vectors; the quantizer snaps each to its nearest codebook entry via argmin (non-differentiable — use straight-through estimator for gradients). The decoder reconstructs the image from quantized codes. Key failure mode: codebook collapse, where most codes go unused. Fix with EMA updates, random restart for dead codes, and k-means initialization.

Codebook Health Metrics

- **Utilization:** % of K codes used over a rolling window. Want >70%. Measures whether codes are alive.
- **Perplexity:** $\exp(\text{entropy of code usage distribution})$. Maximum = K (perfectly uniform). Want >60% of K. Measures whether alive codes are used fairly.
- **Reconstruction images:** the ultimate judge — log to W&B; every 500 steps.

KV-Cache Decode Attention

During autoregressive generation, the transformer processes one new token per step. Standard attention reloads the entire KV-cache from HBM every step — $O(L)$ HBM reads per token, where L is the current sequence length. This is memory-bandwidth bound and worsens linearly with L. A tiled decode attention kernel loads the KV-cache in SRAM-sized tiles and fuses the score computation + weighted sum into one pass, reducing HBM round-trips to $O(L/\text{tile_size})$. This is the core optimization in FlashAttention, PagedAttention (vLLM), and FlashDecoding.

Why Inference > Training Kernels

- Inference is memory-bandwidth bound; training's big ops (GEMM, large attention) are compute-bound and already well-optimized by cuBLAS/cuDNN
- Autoregressive decoding is the bottleneck in every deployed LLM — commercially and technically more relevant
- Speedup scales with sequence length, giving you a more interesting benchmark story than a single number
- Directly motivated by your world model — the imagination loop is pure autoregressive decoding

FID (Fréchet Inception Distance)

Standard generative model quality metric. Passes real and generated images through a pretrained InceptionV3 network, fits a Gaussian to each set of features, and computes the Fréchet distance between the two Gaussians. Lower is better. Requires 10k+ samples to be statistically reliable. Captures perceptual quality; MSE captures pixel accuracy. Report both.